PLONK Prover 분석

개요

PLONK Prover는 영지식 증명을 생성하는 핵심 구성 요소입니다. 이 문서는 `plonk-core/src/proof_system/prover.rs` 에 구현된 Prover의 구조와 증명 생성 과정을 분석합니다.

Prover 구조체 분석

주요 필드

```
pub struct Prover<F, P, PC>
where
F: PrimeField,
P: ModelParameters<BaseField = F>,
PC: HomomorphicCommitment<F>,
{
/// 특정 PLONK 회로에 대한 증명을 생성하는 데 사용되는 Proving Key
pub prover_key: Option<ProverKey<F>>,

/// 회로 설명 (StandardComposer)
pub(crate) cs: StandardComposer<F, P>,
```

```
/// 전처리 단계에서 교환된 메시지를 저장

/// 증명을 만들 때마다 복사됨

pub preprocessed_transcript: Transcript,



_phantom: PhantomData<PC>,

}
```

역할과 기능

1. **ProverKey**: 회로별 증명 생성에 필요한 키 정보
2. **StandardComposer**: 회로의 제약 조건과 변수를 관리
3. **Transcript**: Fiat-Shamir 변환을 위한 상호작용 기록
4. **HomomorphicCommitment**: 다항식 약정 체계 (KZG10, IPA 등)

증명 생성 과정 분석

1. 전처리 단계 (Preprocessing)

```
pub fn preprocess(&mut self, commit_key: &PC::CommitterKey) -> Result<(), Err
```

- 회로가 이미 전처리되었는지 확인
- `StandardComposer` 를 통해 ProverKey 생성
- 전처리된 transcript 저장

2. 주요 증명 생성 단계

`prove_with_preprocessed` 메서드에서 수행되는 단계들:

2.1 도메인 설정 및 초기화

```
let domain = GeneralEvaluationDomain::new(self.cs.circuit_bound())?;
let n = domain.size();
let mut transcript = self.preprocessed_transcript.clone();
```

- 평가 도메인 생성 (FFT를 위한 2의 거듭제곱 크기)
- transcript 복사하여 증명별 독립성 보장

2.2 공개 입력 처리

```
transcript.append(b"pi", self.cs.get_pi());
```

- 공개 입력을 transcript에 추가
- 검증자와 동일한 공개 입력 공유 보장

2.3 증인 다항식 계산 및 약정

```
// 변수를 스칼라로 변환하고 도메인 크기에 맞게 패딩
let pad = vec![F::zero(); n - self.cs.w_l.len()];
let w_l_scalar = &[&self.to_scalars(&self.cs.w_l)[..], &pad].concat();
// ... 다른 와이어들도 동일하게 처리

// 평가 형태에서 계수 형태로 변환 (IFFT)
let w_l_poly = DensePolynomial::from_coefficients_vec(domain.ifft(w_l_scalar));
```

```
// ... 다른 다항식들도 동일하게 처리


// 증인 다항식에 약정
let (w_commits, w_rands) = PC::commit(commit_key, w_polys.iter(), None)?;
```

핵심 과정:

- 회로의 와이어 값들을 다항식으로 변환
- 각 와이어 (left, right, output, fourth)에 대한 다항식 생성
- 다항식 약정 체계를 통해 약정 생성

2.4 룩업 다항식 처리

```
// 테이블 압축 인수 생성
let  zeta = transcript.challenge_scalar(b"zeta");


// 룩업 테이블을 단일 요소 벡터로 압축
let  compressed_t_multiset = MultiSet::compress(&[...], zeta);


// 쿼리 테이블 f 계산
let  compressed_f_multiset = MultiSet::compress(&f_scalars, zeta);
```

룩업 시스템:

- 테이블과 쿼리를 압축하여 효율성 향상
- 룩업 제약 조건 처리를 위한 다항식 생성

2.5 순열 다항식 계산

```
// 순열 챌린지 생성
let beta = transcript.challenge_scalar(b"beta");
let gamma = transcript.challenge_scalar(b"gamma");
let delta = transcript.challenge_scalar(b"delta");
let epsilon = transcript.challenge_scalar(b"epsilon");

// 순열 다항식 계산
let z_poly = self.cs.perm.compute_permutation_poly(...);
```

순열 논증:

- 와이어 간의 순열 관계 증명
- 여러 챌린지를 통한 보안성 확보

2.6 몫 다항식 (Quotient Polynomial) 계산

```
let t_poly = quotient_poly::compute::<F, P>(
&domain,
prover_key,
&z_poly, &z_2_poly,
&w_l_poly, &w_r_poly, &w_o_poly, &w_4_poly,
&pi_poly, &f_poly, &table_poly, &h_1_poly, &h_2_poly,
&alpha, &beta, &gamma, &delta, &epsilon, &zeta,
&range_sep_challenge, &logic_sep_challenge,
```

```
&fixed_base_sep_challenge, &var_base_sep_challenge,
&lookup_sep_challenge,
)?;


// 몫 다항식을 8개로 분할
let  t_i_polys = self.split_tx_poly(n, &t_poly);
```

몫 다항식의 역할:

- 모든 제약 조건의 만족을 하나의 다항식으로 통합
- 차수 제한을 위해 8개 부분으로 분할

2.7 선형화 다항식 (Linearisation Polynomial) 계산

```
let  z_challenge = transcript.challenge_scalar(b"z");


let (lin_poly, evaluations) = linearisation_poly::compute::<F, P>(
&domain, prover_key,
&alpha, &beta, &gamma, &delta, &epsilon, &zeta,
&range_sep_challenge, &logic_sep_challenge,
&fixed_base_sep_challenge, &var_base_sep_challenge,
&lookup_sep_challenge, &z_challenge,
&w_l_poly, &w_r_poly, &w_o_poly, &w_4_poly,
&t_i_polys[0], &t_i_polys[1], &t_i_polys[2], &t_i_polys[3],
&t_i_polys[4], &t_i_polys[5], &t_i_polys[6], &t_i_polys[7],
```

```
&z_poly, &z_2_poly, &f_poly, &h_1_poly, &h_2_poly, &table_poly,

)?;
```

선형화의 목적:

- 검증자가 효율적으로 검증할 수 있도록 다항식을 선형 결합으로 표현
- 특정 점에서의 평가값들을 포함

2.8 배치 열기 증명 (Batch Opening Proof)

```
// 집계 증인 (Aggregate Witness) 계산

let  aw_challenge: F = transcript.challenge_scalar(b"aggregate_witness");

let  aw_opening = PC::open(commit_key, ...)?;


// 시프트된 집계 증인 (Shifted Aggregate Witness) 계산

let  saw_challenge: F = transcript.challenge_scalar(b"aggregate_witness");

let  saw_opening = PC::open(commit_key, ...)?;
```

배치 열기의 효율성:

- 여러 다항식의 열기를 하나의 증명으로 통합
- 검증 시간과 증명 크기 최적화

3. 최종 증명 구조체 생성

```
Ok(Proof {

a_comm: w_commits[0].commitment().clone(),

b_comm: w_commits[1].commitment().clone(),
```

```
c_comm: w_commits[2].commitment().clone(),

d_comm: w_commits[3].commitment().clone(),

z_comm: saw_commits[0].commitment().clone(),

f_comm: f_poly_commit[0].commitment().clone(),

h_1_comm: h_1_poly_commit[0].commitment().clone(),

h_2_comm: h_2_poly_commit[0].commitment().clone(),

z_2_comm: z_2_poly_commit[0].commitment().clone(),

t_1_comm: t_commits[0].commitment().clone(),

// ... 나머지 약정들

aw_opening,

saw_opening,

evaluations,

})
```

보조 메서드 분석

1. `split_tx_poly` - 몫 다항식 분할

```
fn  split_tx_poly(&self, n: usize, t_x: &DensePolynomial<F>) -> [DensePolynomial
```

- 몫 다항식을 8개의 n-크기 다항식으로 분할
- SRS(Structured Reference String) 크기를 회로 크기에 선형적으로 제한

2. `to_scalars` - 변수를 실제 값으로 변환

```
fn  to_scalars(&self, vars: &[Variable]) -> Vec<F>
```

- Variable 인덱스를 실제 필드 원소 값으로 매핑
- 증인 다항식 생성에 필요

3. 상태 관리 메서드들

- `clear_witness()` : 증인 데이터만 초기화 (동일 회로로 다른 증명 생성 시)
- `clear()` : 모든 데이터 초기화 (다른 회로로 증명 생성 시)
- `key_transcript()` : transcript에 추가 시드 정보 추가

성능 및 보안 고려사항

성능 최적화

1. **FFT 활용**: 다항식 연산을 평가 도메인에서 수행
2. **배치 처리**: 여러 약정과 열기를 배치로 처리
3. **메모리 관리**: 증인 데이터 재사용을 위한 clear 메서드 제공

보안 특성

1. **Fiat-Shamir 변환**: 상호작용을 비상호작용으로 변환
2. **챌린지 분리**: 서로 다른 제약 조건에 독립적인 챌린지 사용
3. **영지식성**: 증인 정보를 노출하지 않는 증명 생성

요구사항 충족 확인

- **요구사항 3.1**: Prover의 구조와 역할 파악 ✓

- **요구사항 3.3**: 증명 생성 과정의 각 단계 분석 ✓

이 분석을 통해 PLONK Prover가 어떻게 회로의 만족성을 영지식으로 증명하는지, 그리고 각 단계에서 어떤 암호학적 기법들이 사용되는지 이해할 수 있습니다.